The Starter Guide for Snowflake™

A 5 minute guide to help you start mastering Snowflake.

Few tools have made a bigger splash in the data world than Snowflake.

Even if your company doesn't use it, it's a platform you should **absolutely be familiar with** as a data engineer. *But why?*

In this guide, you'll learn:

- What Snowflake is
- Why businesses love it
- 7 key concepts to know as a developer
- 5 best practices to consider

Let's dive in.

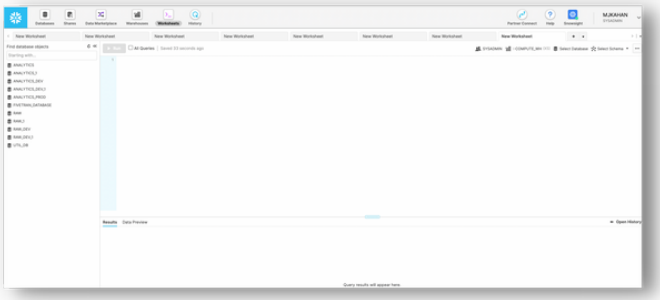
WHAT IS SNOWFLAKE?

*By definition, Snowflake is a **cloud-based data warehouse** provider.*

You'll create database objects, load data, write queries and much more. Just as you would with most other database providers.

But it comes with a slick user interface, top-notch capabilities and is completely **hosted on the cloud**.

That means everything is web-based and there is **no desktop application to install**.



The main Snowflake developer interface

It's main use case is for **data warehousing** and storing massive amounts of data for **analytical purposes**.

As opposed to a transactional (row-based) database used as the backend of an application.

It primarily competes with other cloud database products such as Amazon Redshift, Google BigQuery and Azure Synapse.

But what makes Snowflake so appealing?

WHY DO BUSINESSES LOVE IT?

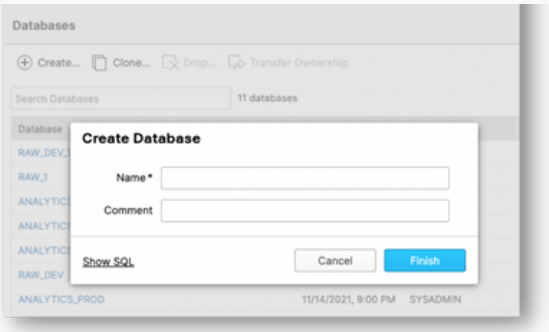
*The two biggest reasons people choose Snowflake are for its **simplicity** and **architecture**.*

Simplicity

Setup / Upgrades

Historically, setting up new database servers required many steps and a lot of planning before it could be ready for use.

But because Snowflake is completely cloud-based, you can quickly spin up a brand new database server **in minutes** with just a few clicks.

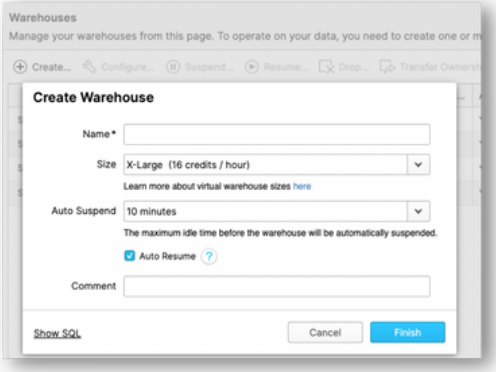


Easily add more databases to your server

But what if you underestimated and need more space or better performance?

Simply add on to your existing server and you'll be **billed based on usage**.

No need to have your team go to a data center or run a manual upgrade. You'll never need to worry about running out of storage or not having enough available performance.
(assuming you'll willing to spend the $$$)



Add more compute power
(called Warehouses in SF)

Ease of Use / Familiarity

Snowflake is also incredibly easy to use, even if you're brand new. It simply *just works*.

As a developer, the interface will feel familiar and the object explorer is clean. And the best part, you can write standard ANSI SQL just as you would in most other database tools.

Example documentation

There are **also many Snowflake-specific SQL functions** but everything is well-documented (with examples) to make sure you use them correctly.

This all helps to reduce the learning curve and allows new developers to start performing basic operations right away.

Check out the full documentation site here.

Architecture

Simplicity is great, but the underlying architecture is what truly sets it apart.

3-Layered Design

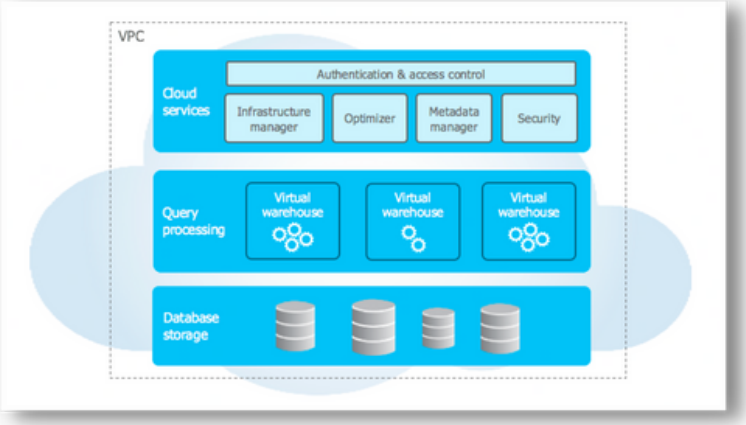
The three main layers are: Database Storage, Query Processing *(aka Compute)* and Cloud Services *(roles, metadata, etc.)*.

Most important here is the fact that **storage and compute are separated**.

This is a major difference from traditional on-premise servers where both are typically pulled from the same machine.

Keeping them separate allows both concepts to scale independently and for the compute power to be dynamically adjusted based on demand.

All of this is possible because of Snowflake's cloud-first design which is geared towards individual services *(think Azure or AWS services)*.

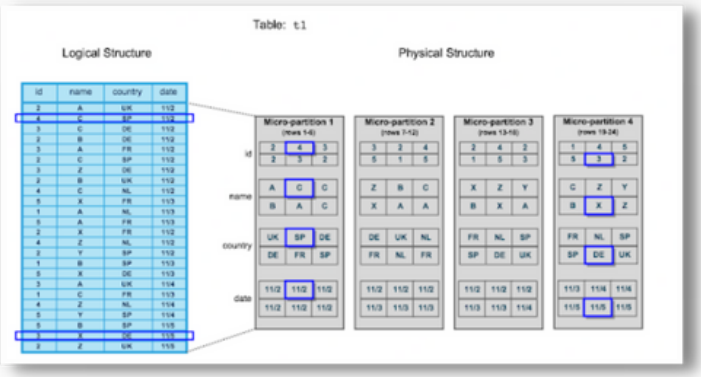


The 3 layers of Snowflake

Micro-Partitions

Whenever data is loaded, Snowflake will **automatically** compress it (to save space), determine the optimal micro-partition (how it is physically stored) and record the metadata.

The technical details of how this works are outside the scope of this guide, but what's important to note is that **Snowflake automatically handles all of this** behind the scenes for you and allows for **highly efficient query processing.**



Comparing Snowflake's logical vs physical data structure

For example, Snowflake will know the min/max values for all rows in a column based on the micro-partition metadata.

Therefore, a query to select the min/max of a specific column only needs to be a simple metadata operation (rather than actually querying the whole table).

This part can get a bit technical, so feel free to read more about this concept here.

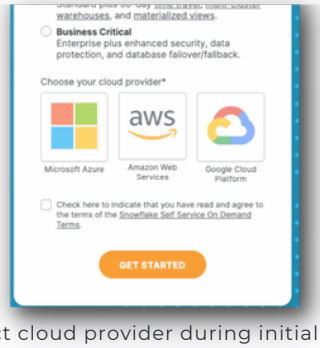
KEY CONCEPTS TO KNOW

There are a lot of technical reasons why Snowflake is so powerful, but here are some of the most important ones to understand as a developer:

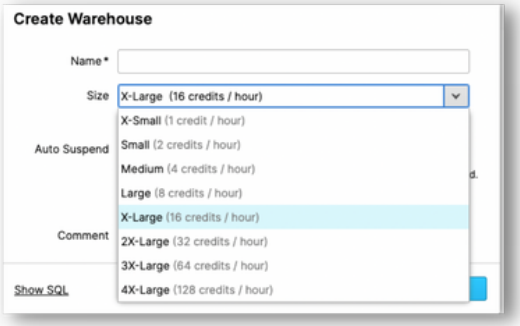
Cloud Agnostic

Technically speaking, Snowflake uses AWS/Azure/GCP services under the hood and you get to select which one to use during setup.

This means you are not tied down to any single cloud platform and can avoid potential conflicts of interest.



Select cloud provider during initial setup



Adjust warehouse size based on needs

Warehouses

Snowflake refers to its **compute objects** as “warehouses”, which can be a bit confusing because it is not referring to how data is stored. You can create multiple warehouses with different sizes (aka more or less computing power).

Warehouses are sized like T-Shirts (S,M,L,XL) and can be dynamically changed to fit your computational needs.

For example, you might use a larger warehouse to handle a large batch process once an hour but use a smaller one for ad-hoc querying.

"Variant" Data type

Modern data sources often consist of semi-structured formats like JSON or XML. In Snowflake, you can store this type of data in a single column as a variant data type. You can then use built-in Snowflake functions to “flatten” the data into a cleaner tabular form, if needed.

Stages

A stage in Snowflake is an intermediate space where you can upload files and use the COPY command to load or unload tables. A common example is an External Stage, which allows you to reference a storage location in another cloud provider.

Let's say you already have a process loading data into AWS S3 bucket and now want to move it to Snowflake for further analytics. You can use this concept to easily bridge that gap.

Shares

Like the name implies, Snowflake Data Sharing enables sharing selected objects in a database in your account with other Snowflake accounts. The data itself is not copied, but rather you can give another user “read only” access to the data.

Think of it like providing read-only access to a shared doc, but instead of a doc it’s a database object.

MAIN CONCEPTS

Continued

Zero-Copy Clones

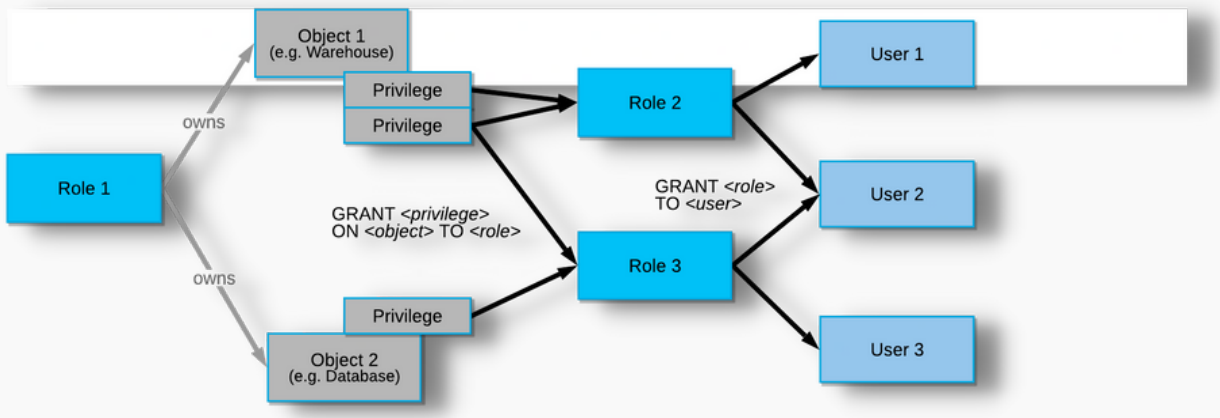
Snowflake allows you to create a “copy” or clone of an existing table without duplicating the data. The new table will point to the same underlying data records *(as of the time of cloning)* but then manage any further changes separately. For example, a record added (or deleted) in a cloned object will not impact the original table.

This flexibility is a direct result of the "micro-partitioning" architecture that was referenced earlier.

Note: Cloning is not intended to be a replacement for a data backup strategy.

Role-Based Security

Roles and security are part of the "Cloud Services" layer and play a major role in Snowflake. Users are not typically given direct permission to objects in your db *(ex. Tables, schemas)*. Rather, ROLES are given permissions, and then **users are assigned to roles**.



Overview of roles, objects and users

BEST PRACTICES TO CONSIDER

Manage infrastructure with code

Roles and objects can get out of hand quickly. But a code-based tool like Terraform allows you to version control your setup and quickly set up your environment the exact way you want it at any time.

Build wider tables, limit joins

As a columnar database, Snowflake is designed to handle a lot of data but is not optimized for a lot of row-based operations (such as joins).

It is incredibly powerful and will be able to handle most of what you throw at it, but try to limit the number of joins in a query for best results.

Leverage CTEs for organization

Modern execution plans are really smart and can identify what you need (and don't need).
For that reason, you can leverage CTEs to better organize and even "import" full tables without impacting performance.

You can read more about this topic here .

Auto-suspend warehouses

Computation is by far the most expensive part of Snowflake and none of us want to pay more if we don't have to. A great feature is to "auto-suspend" your warehouses so they don't rack up costs during idle times.

Separate raw (source) from transformed (analytics)

This one is a personal preference, but I feel the combination of role-based permissions, storage capabilities and transformation tools (like dbt) lends itself well to separating these concepts.

For example, have one database for your raw data *(each source gets a schema)*, and a separate one for analytics *(staging, data warehouse, reporting)*. Of course there are other ways to approach it, but if you are looking for a suggestion, that would be mine.

Thanks For Reading!

I hope you found this guide helpful. There is **so much more** to learn about Snowflake, but hopefully now you have a better appreciation for its purpose and the type of things to expect as a developer.

Cheers,
Mike